

STEP 2

Data understanding and validation

Know what each column means, decide which ones the model may use, and stop bad data at the door.

The dataset

8,000 daily inventory records (Jan-Dec 2025) for four business types and six food categories. It is synthetic but generated from a causal model: demand → purchasing → sales → leftover stock → waste. About 3% of values are missing in five columns, and about 1% of order quantities are outliers.

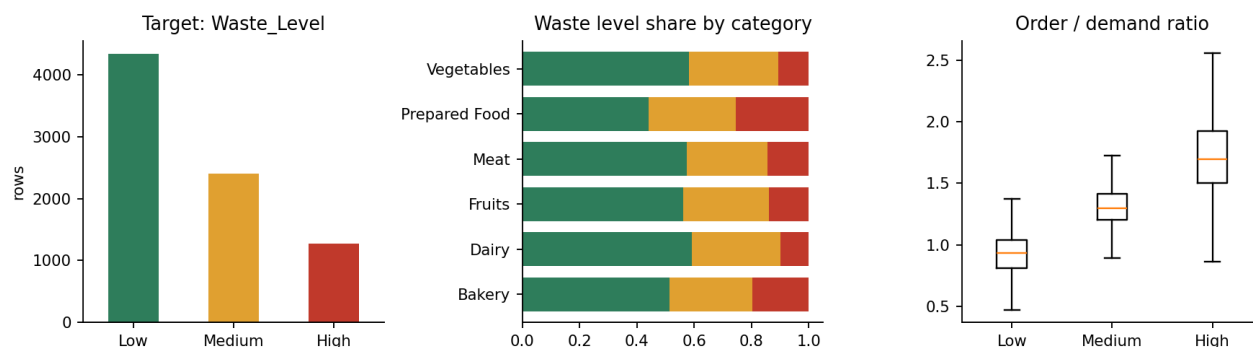


Figure 2 - Left: the target is imbalanced (54% Low, 30% Medium, 16% High). Middle: share of each waste level by category. Right: ordering more than demand is strongly linked to higher waste.

Target and inputs

Role	Columns	Reason
Target	Waste_Level (Low / Medium / High)	What we predict
Inputs (known in the morning)	Date, Business_Type, Food_Category, Purchase_Quantity, Shelf_Life_Days, Storage_Temperature, Daily_Demand, Promotion, Weather	Available when the order is placed
Excluded: leakage	Units_Sold, Remaining_Stock, Waste_Quantity, Waste_Reason, Donation_Made	Only known after the day ends, or computed from the waste itself
Excluded: ID	Inventory_ID	Unique per row, carries no signal

Data leakage

If a column that is only known *after* the outcome is used as an input, the model looks excellent in testing and fails in real use. The notebook demonstrates this: a simple tree using Waste_Quantity / Purchase_Quantity reaches **96.7% accuracy**, but nobody knows the waste quantity in the morning. The leakage columns are listed in config.py and a test fails if any of them ever appears in the feature list.

Assumption

Daily_Demand is borderline: the dataset defines it as the demand that actually happened. This project treats it as the business's demand forecast, which is a reasonable assumption, and the README says so openly. Stating assumptions like this is part of good ML practice.

Validation rules

`validate()` in `foodwaste/data.py` runs before training *and* before every prediction. It collects every problem and raises one clear error:

Rule	Example of rejected data
All required columns exist	File without Food_Category
Categories are from the allowed list	Food_Category = 'Seafood'
Purchase_Quantity > 0	Purchase_Quantity = -10
Shelf_Life_Days $\geq$ 1, Daily_Demand $\geq$ 0	Shelf_Life_Days = 0
Date parses as a date	Date = 'not-a-date'
Target has no missing values (training only)	Empty Waste_Level

```
>>> validate(bad)
DataValidationError: Food_Category: unknown values ['Seafood']; Purchase_Quantity must be > 0
```

Why it matters

Most production ML failures are data failures, not model failures. A validation layer turns silent garbage-in/garbage-out into a loud, explainable error.